Greedy

Greedy

Algorithms for optimization problems typically go through a sequence of steps, with a set of choices at each step.

A **greedy algorithm** always makes the choice that looks best at the moment.

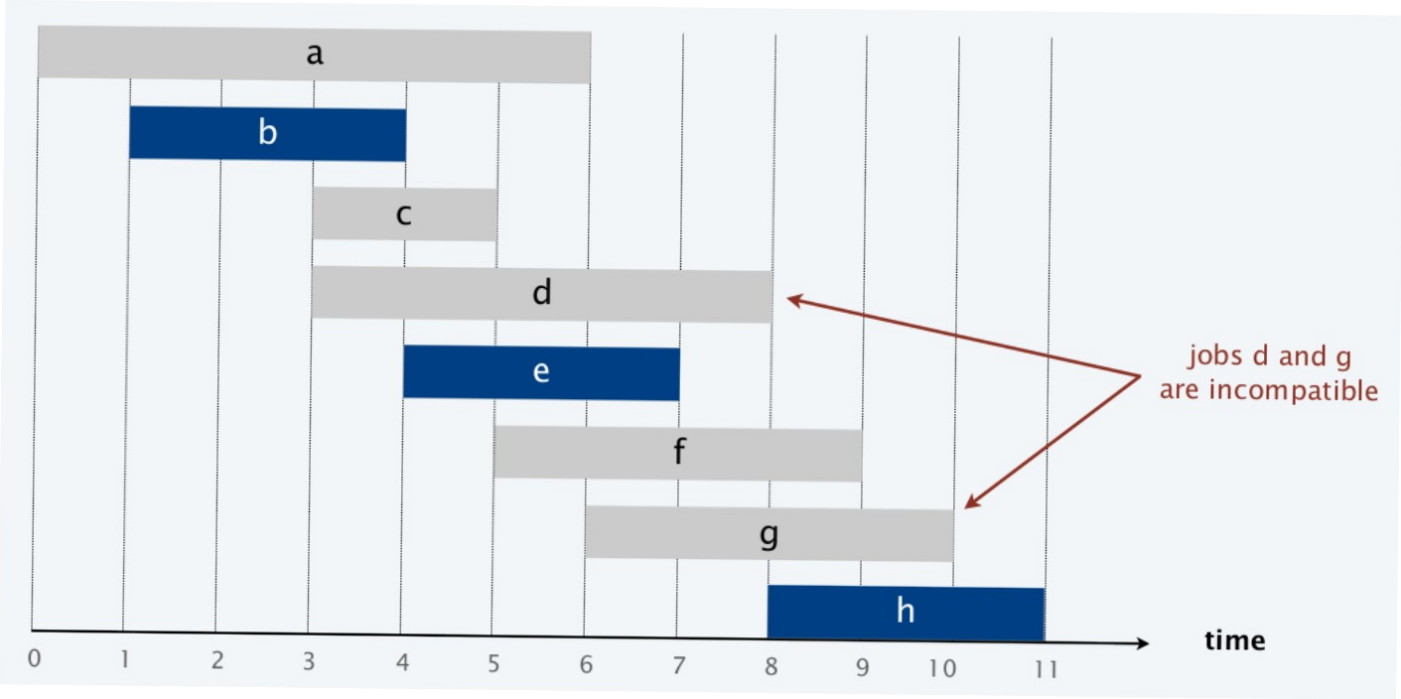
That is, it makes a **locally optimal choice** in the hope that this choice will lead to a **globally optimal solution**.

Question: Does greedy algorithm always yield optimal solution?

Interval Scheduling

Input: A set of jobs with start and finish times,

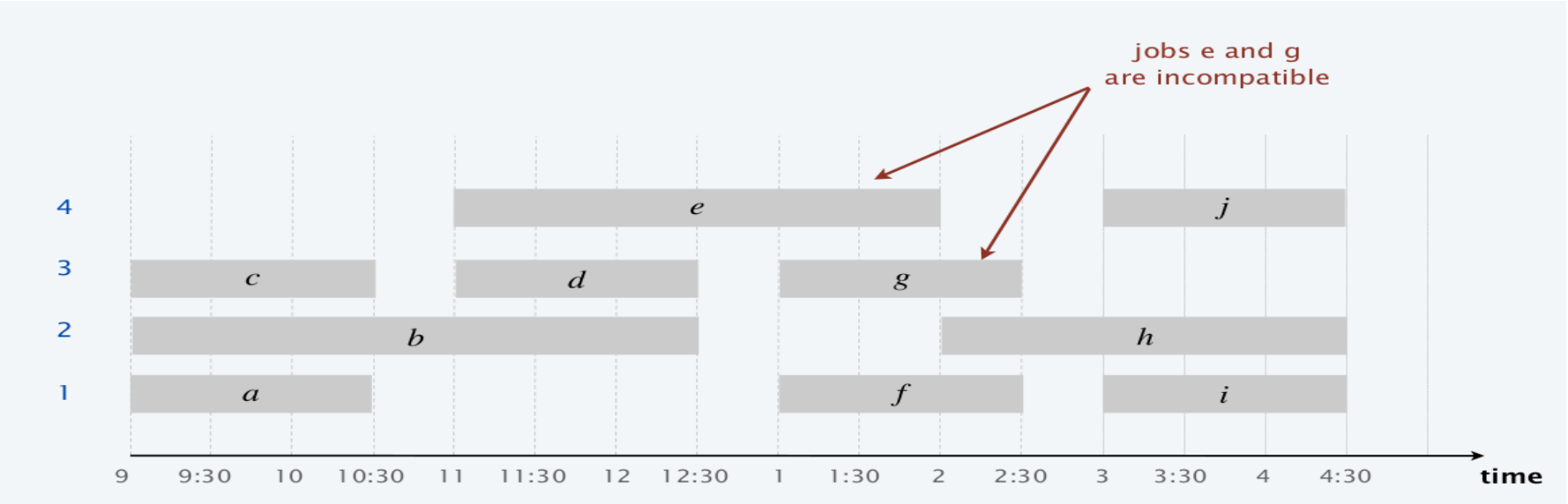
Question: find the maximum subset of mutually compatible (non-overlapping) jobs.



Interval Partitioning

Input: A set of lectures with start and finish times,

Question: find the minimum number of classrooms to schedule all the lectures so that no two lectures happen at the same time in the same room.



Huffman Coding

Huffman Coding

Input: A large file, say 100,000-character data file.

Goal: Design a method to store the data compactly.

Huffman Coding

Input: A large file, say 100,000-character data file.

Goal: Design a method to store the data compactly.

Using binary code, where each character is represented by a unique binary string called **Codeword**

Huffman Coding

Input: A large file, say 100,000-character data file.

Goal: Design a method to store the data compactly.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword						

Huffman Coding

Input: A large file, say 100,000-character data file.

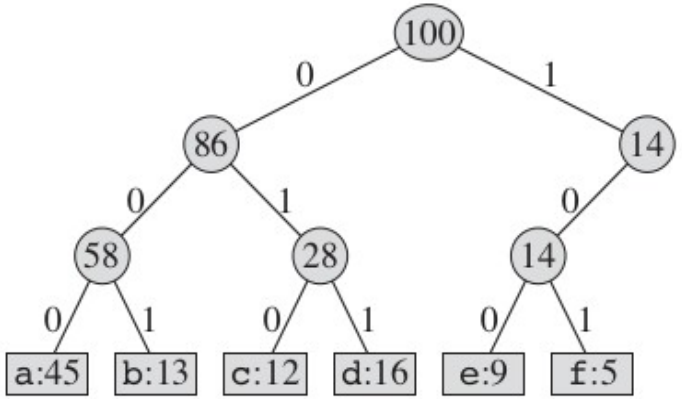
Goal: Design a method to store the data compactly.

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

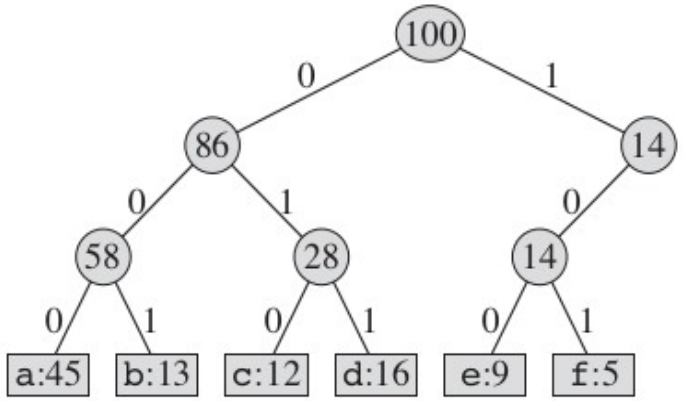
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

How does such an encoding look like?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

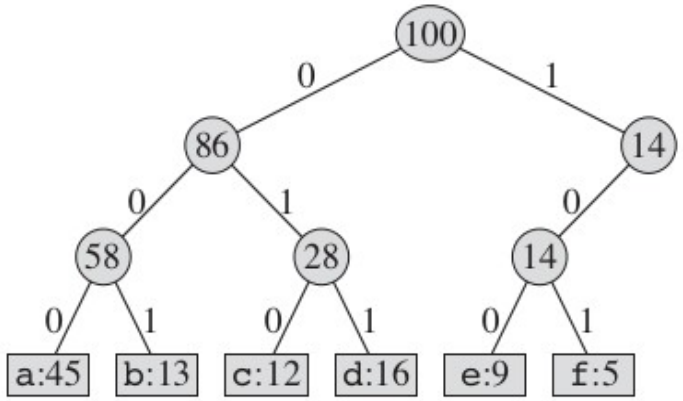


(a)



	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

#bits?



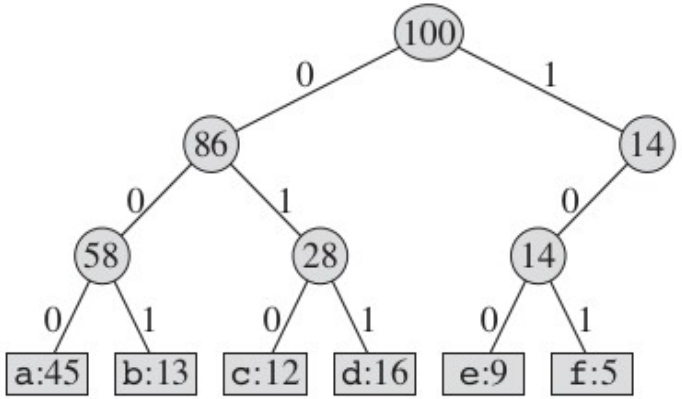
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101

#bits?

(45+13+12+16+9+5)*3

= 300 K

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101



(a)

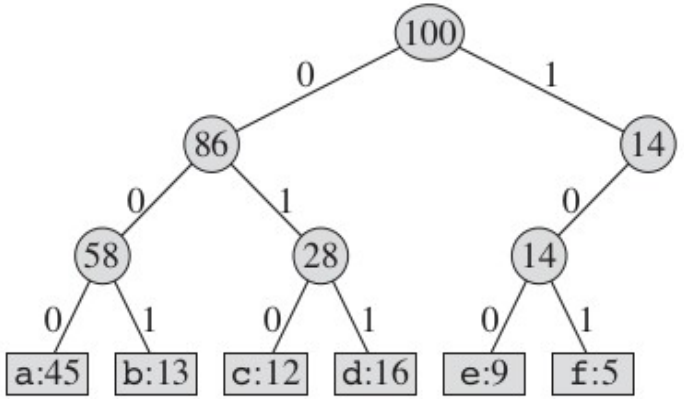
Can we do better?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

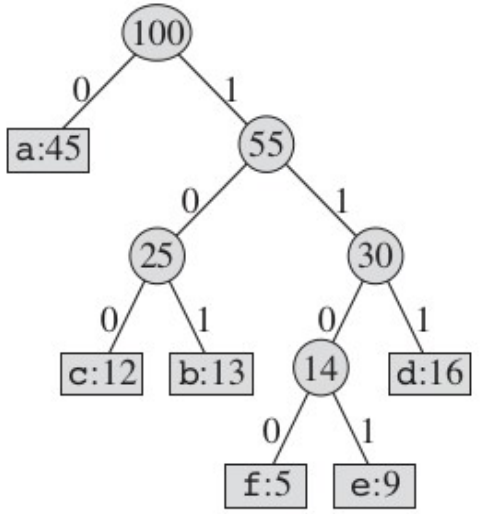
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

How does the encoding look?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100



(a)



(b)

Huffman Coding

#bits?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Huffman Coding

#bits?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Fixed-length codeword

(45+13+12+16+9+5)*3

= 300 K

Huffman Coding

#bits?

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Fixed-length codeword

(45+13+12+16+9+5)*3

= 300 K

45*1 + 13*3 + 12*3 + 16*3 + 9*4 + 5*4

= 224 K

Prefix codes

Codes in which no codeword is a prefix of some other codeword.

Such codes are called **prefix codes**.

Prefix codes

Codes in which no codeword is a prefix of some other codeword.

Such codes are called **prefix codes**.

- Encoding is done by concatenation of codewords.
- Prefix codes are desirable because they simplify decoding. Why?

How is it beneficial?

Prefix codes

Codes in which no codeword is a prefix of some other codeword.

Such codes are called **prefix codes**.

- Encoding is done by concatenation of codewords.
- Prefix codes are desirable because they simplify decoding. Why?

- (Since no codeword is a prefix of any other, the codeword that begins an encoded file is unambiguous.)

Applications

- Text-file compression
- ZIP and GZIP compression
- PNG and JPEG image compressions
- Audio and Video compressions
- Fax transmission
- Data transmission and storage

Cost

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

#bits required for the encoding of T corresponding to the prefix code?

B(T)=

Cost

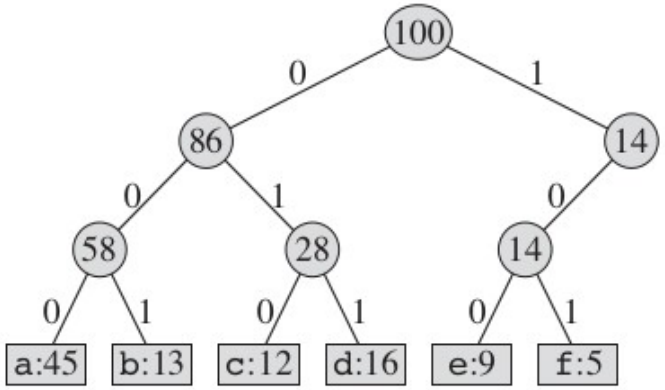
	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

#bits required for the encoding of T corresponding to the prefix code?

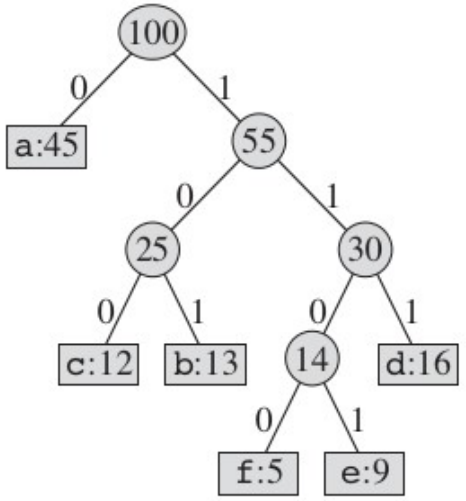
$B(T) = \Sigma_{c \epsilon C} c.freq * d_T(c)$.

Algorithm

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100



(a)



(b)

Greedy Step?

Greedy Strategy -1

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Greedy Strategy

Pick the character with the greatest frequency and make it the left node of the rooted-subtree.

Does this work?

Greedy Strategy -1

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

Greedy Strategy

Pick the character with the greatest frequency and make it the left node of the rooted-subtree.

Consider the input

a:10, b:6, c:5, d:7, e:8

Is it OPTIMAL?

Greedy Strategy -2

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

While (there are at least two characters)

- Find two characters with the least frequencies.
- Merge them

Greedy Strategy -2

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

While (there are at least two characters)

- Find two characters with the least frequencies.
- Merge them

How many times is this step executed?

Greedy Strategy -2

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

HUFFMAN(C)

1 $n = |C|$

2 $Q = C$

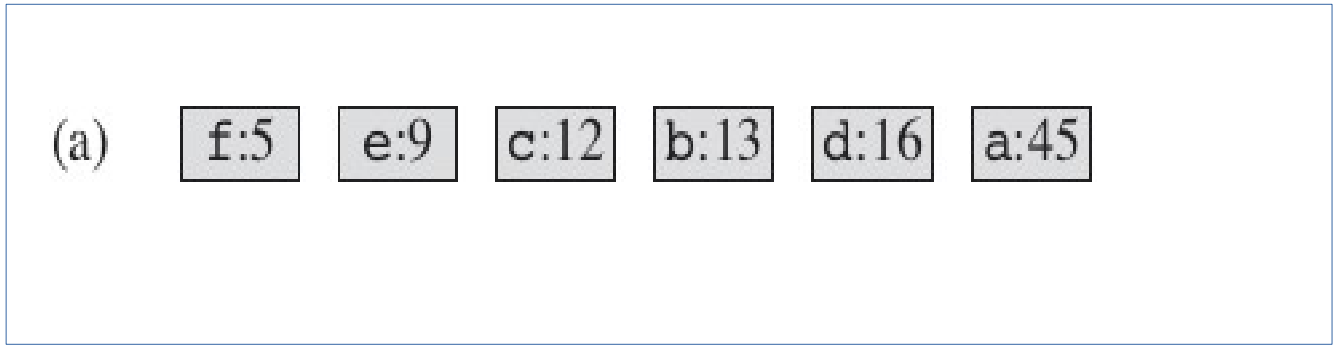
Greedy Strategy -2

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

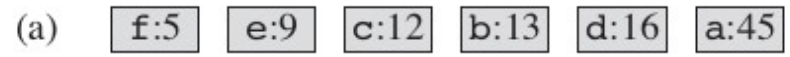
```
HUFFMAN(C)
1  n = |C|
2  Q = C
3  for i = 1 to n − 1
4       allocate a new node z
5       z.left = x = EXTRACT-MIN(Q)
6       z.right = y = EXTRACT-MIN(Q)
7       z.freq = x.freq + y.freq
8       INSERT(Q, z)
9  return EXTRACT-MIN(Q)      // return the root of the tree
```

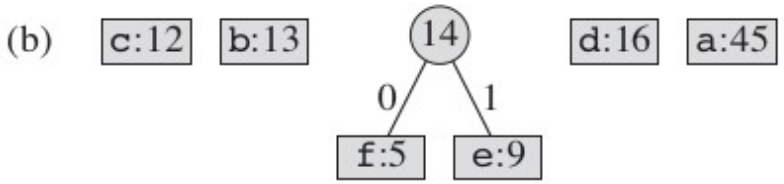
Illustration

	a	b	c	d	e	f
Frequency (in thousands)	45	13	12	16	9	5
Fixed-length codeword	000	001	010	011	100	101
Variable-length codeword	0	101	100	111	1101	1100

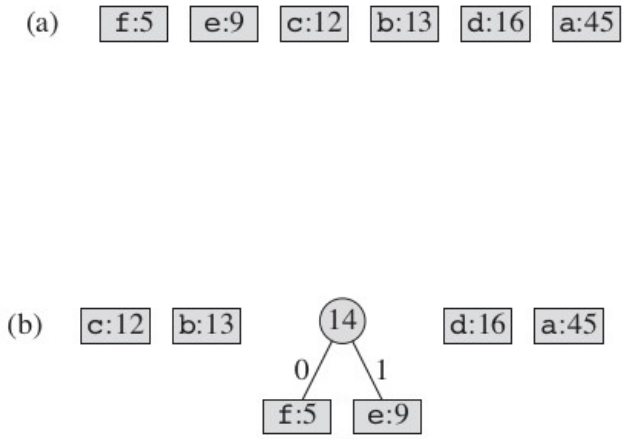


Illustration

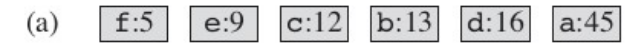


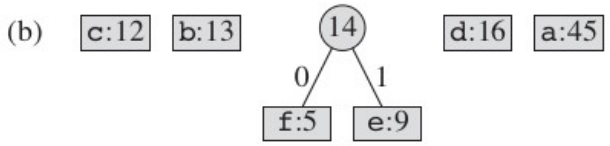


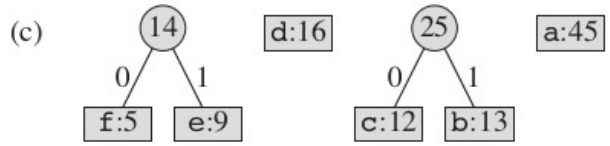
Illustration

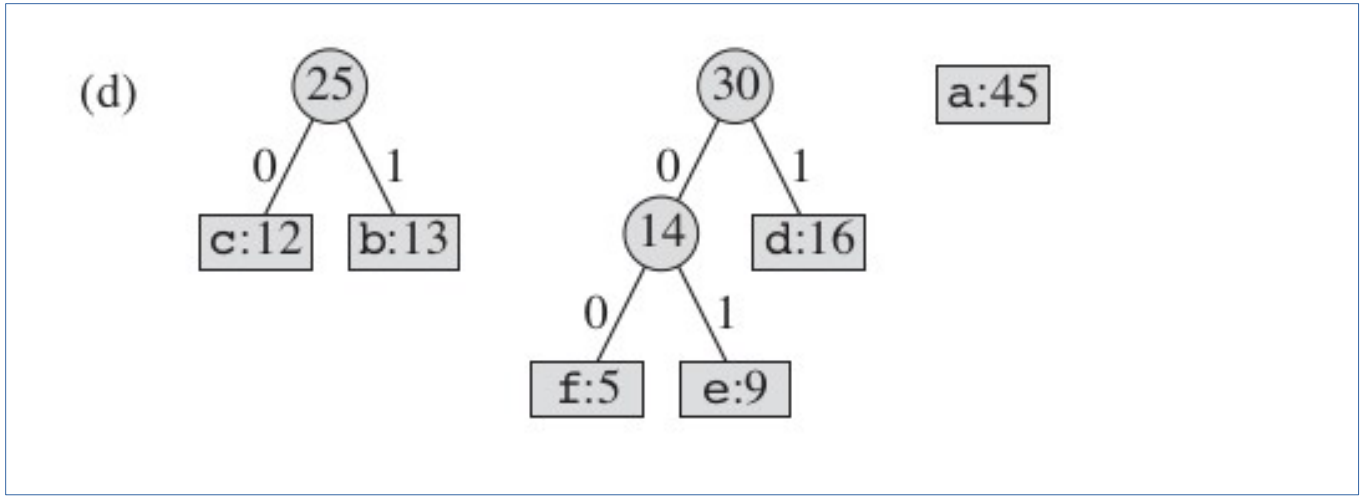


Illustration

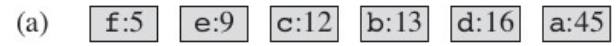


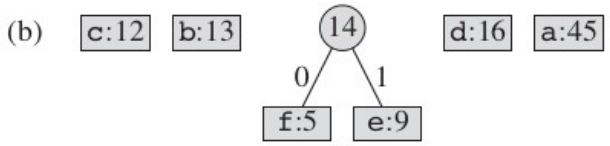


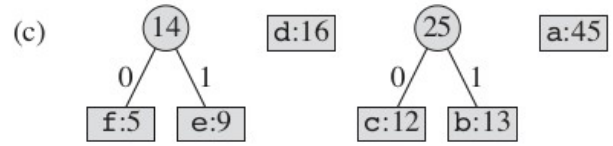


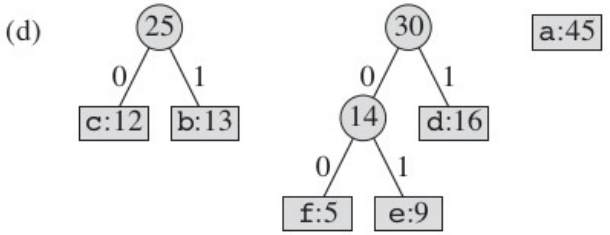


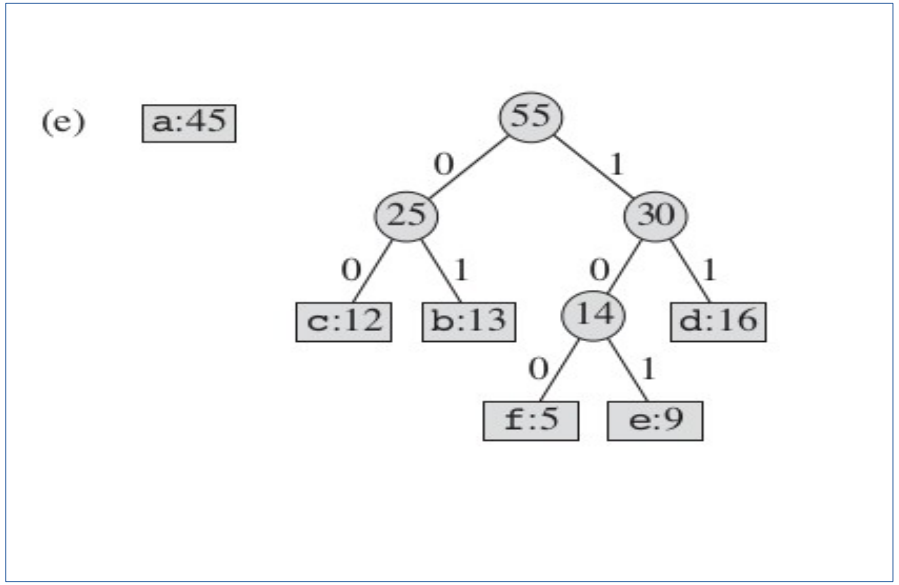
Illustration



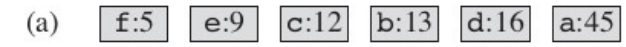


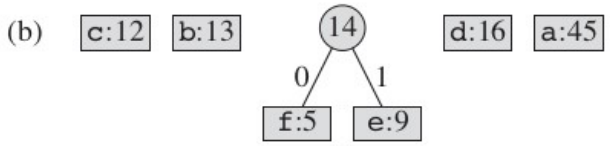


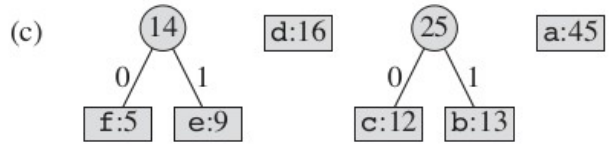


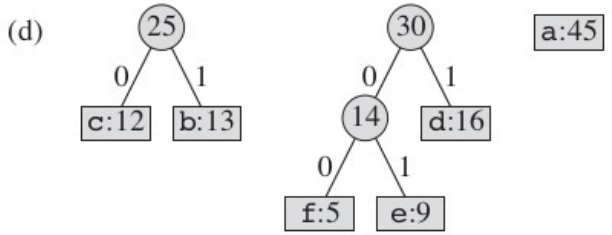


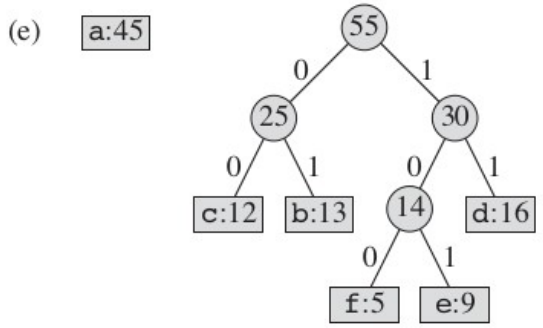
Illustration

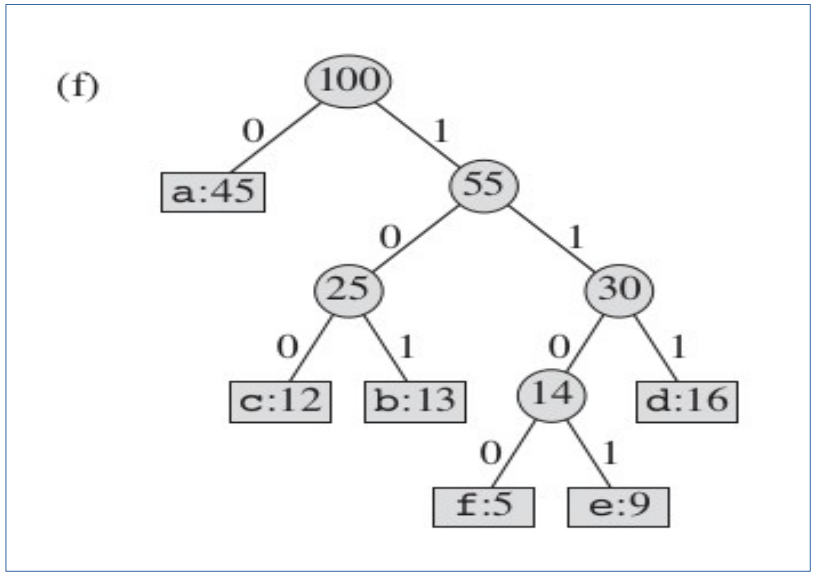












Correctness

Fact: prefix code can always achieve the optimal data compression among any character code.

While (there are at least two characters)

- Find two characters with the least frequencies.
- Merge them

Correctness

While (there are at least two characters)

- Find two characters with the least frequencies.
- Merge them

How to show optimality for our algorithm?

Greedy (3-steps) : general framework

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

2) Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**. [Greedy Choice Property]

3) Demonstrate **optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem. [Optimal Substructure Property]

Correctness

While (there are at least two characters)

- Find two characters with the least frequencies.
- Merge them

1. Two minimum frequency characters are the siblings.

2. Replace the resultant tree with a character and a frequency and repeat the process

Greedy choice

Subproblem

Greedy choice

Two minimum frequency characters are the siblings.

Subproblem

Replace the resultant tree with a character and a frequency.

Characters C C’= C – {x,y} + {z}

z.freq = x.freq + y.freq

Greedy Choice (Greedy is safe)

Two minimum frequency characters are at the siblings.

Two characters of minimum frequency should be the siblings in some optimal prefix code.

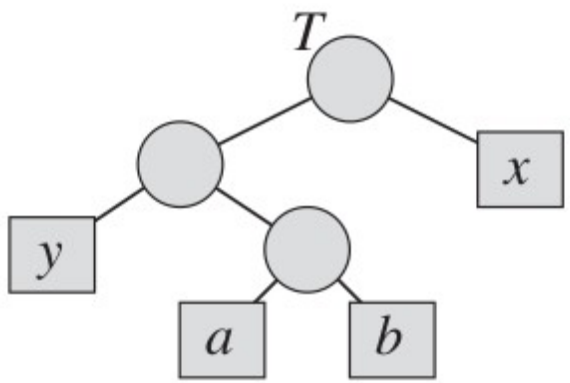
Greedy is safe (Lemma 1)

Two minimum frequency characters are at the siblings.

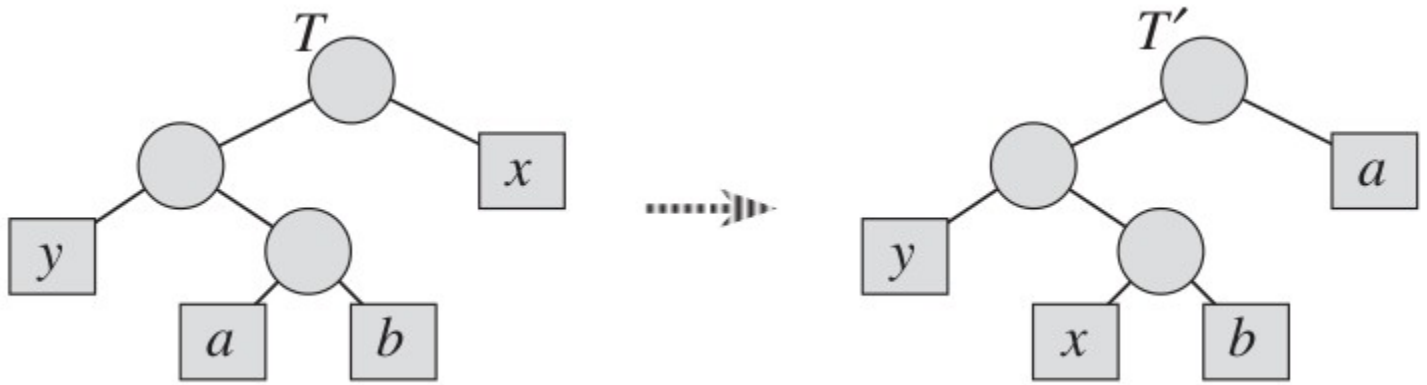
Two characters of minimum frequency should be the siblings in some optimal prefix code.

Let C be an alphabet in which each character $c \in C$ has frequency $c.freq$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.

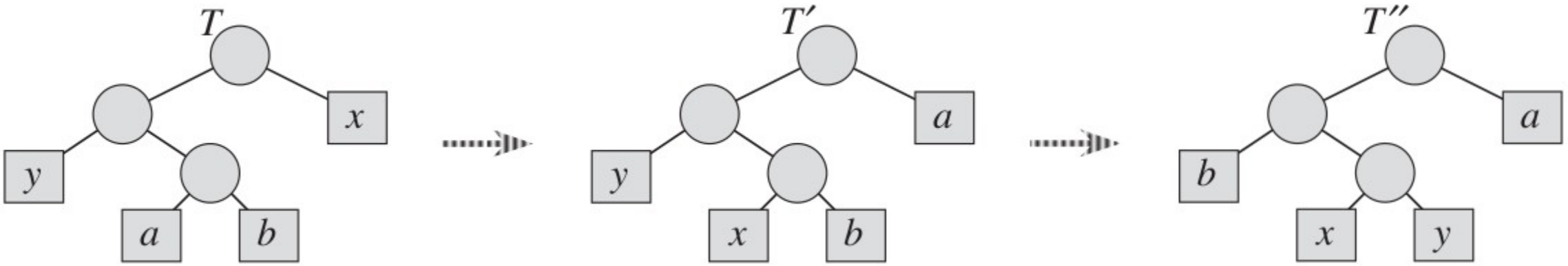
Let C be an alphabet in which each character $c \in C$ has frequency $c.freq$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.



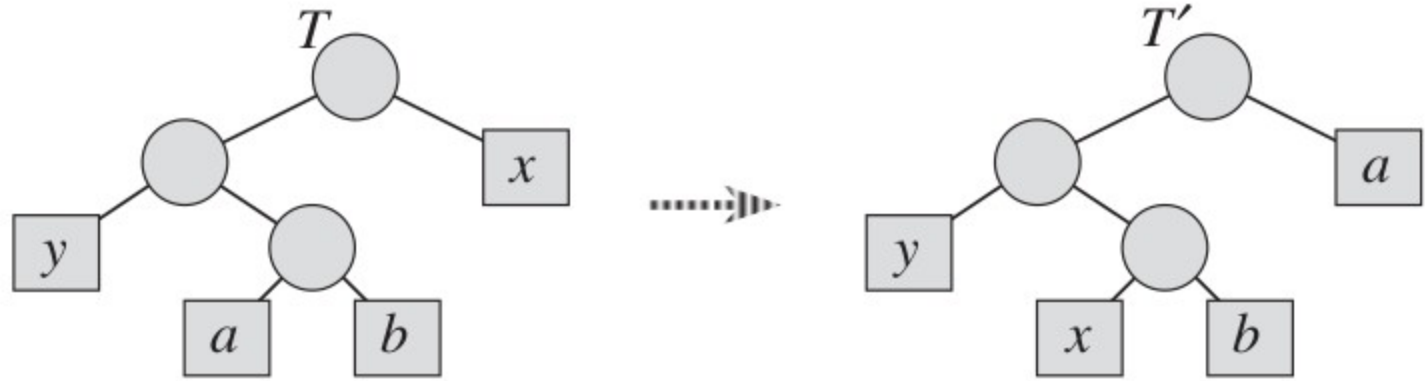
Let C be an alphabet in which each character $c \in C$ has frequency $c.freq$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.



Let C be an alphabet in which each character $c \in C$ has frequency $c.freq$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.



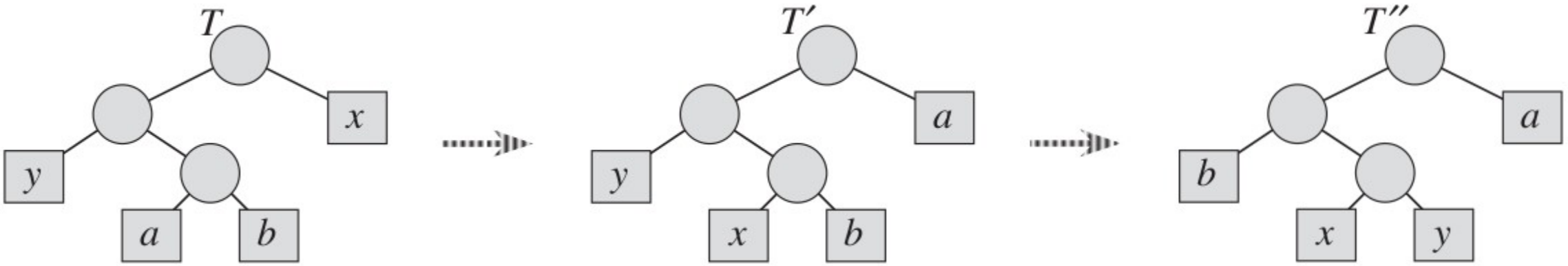
Can you compare the costs for the trees T and T'?

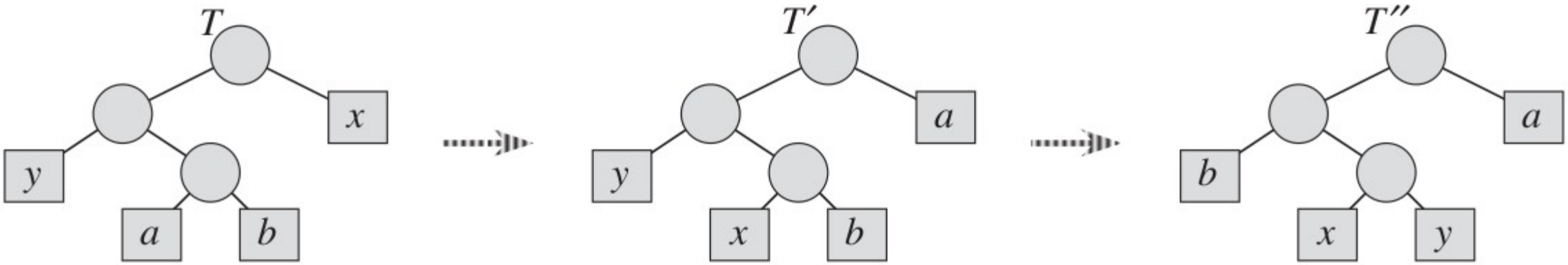


$$
\begin{aligned}
B(T) - B(T') & \\
&= \sum_{c \in C} c.freq \cdot d_T(c) - \sum_{c \in C} c.freq \cdot d_{T'}(c) \\
&= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_{T'}(x) - a.freq \cdot d_{T'}(a) \\
&= x.freq \cdot d_T(x) + a.freq \cdot d_T(a) - x.freq \cdot d_T(a) - a.freq \cdot d_T(x) \\
&= (a.freq - x.freq)(d_T(a) - d_T(x)) \\
&\geq 0 \,,
\end{aligned}
$$

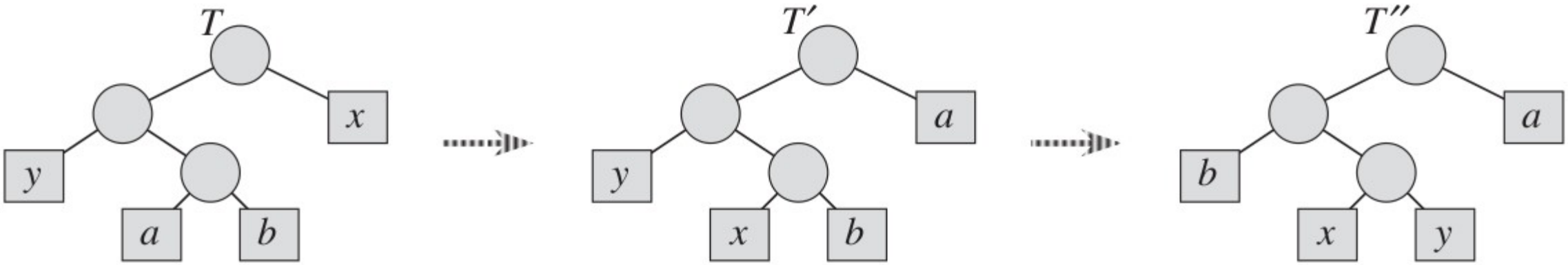
Let C be an alphabet in which each character $c \in C$ has frequency $c.freq$. Let x and y be two characters in C having the lowest frequencies. Then there exists an optimal prefix code for C in which the codewords for x and y have the same length and differ only in the last bit.

What happens if x.freq=b.freq ?





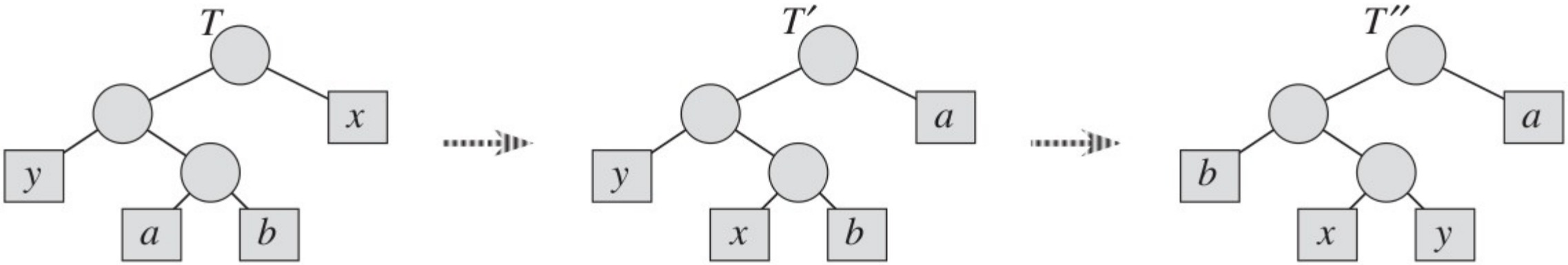
B(T')-B(T'') ≥ ??



$B(T')-B(T'') \geq 0$

$B(T)-B(T') \geq 0$

$B(T) \geq B(T') \geq B(T'')$



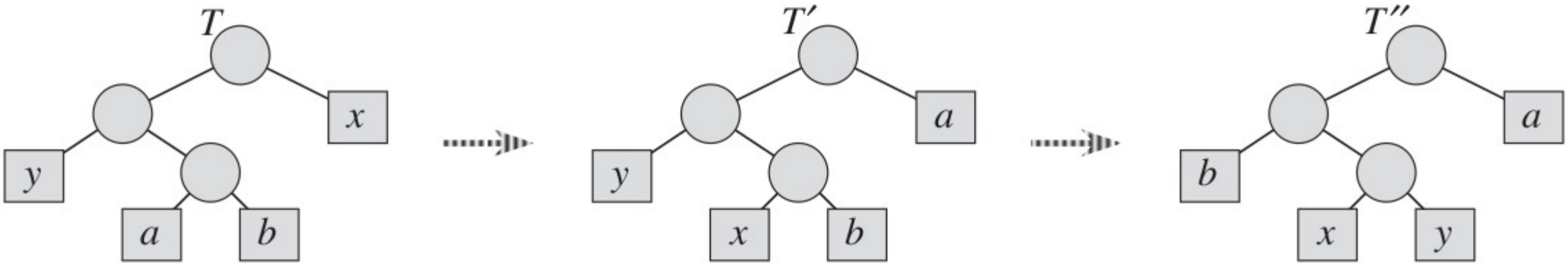
$B(T')-B(T'') \geq 0$

$B(T)-B(T') \geq 0$

$B(T) \geq B(T') \geq B(T'')$

Since T is optimal, what can we infer about T'' ?

$B(T) = B(T'')$



$B(T')-B(T'') \geq 0$

$B(T)-B(T') \geq 0$

$B(T) \geq B(T') \geq B(T'')$

GREEDY CHOICE is SAFE.

Greedy (3-steps) : general framework

1) Cast the optimization problem as one in which we make a choice and are **left with one subproblem to solve**.

2) Prove that there is always an optimal solution to the original problem that makes the greedy choice, so that the **greedy choice is always safe**. [Greedy Choice Property]

3) Demonstrate **optimal substructure** by showing that, having made the greedy choice, what remains is a subproblem with the property that if we combine an optimal solution to the subproblem with the greedy choice we have made, we arrive at an optimal solution to the original problem. [Optimal Substructure Property]

Optimal Substructure (Lemma 2)

Let C be a given alphabet with frequency $c.freq$ defined for each character $c \in C$. Let x and y be two characters in C with minimum frequency. Let C' be the alphabet C with the characters x and y removed and a new character z added, so that $C' = C - \{x, y\} \cup \{z\}$. Define f for C' as for C, except that $z.freq = x.freq + y.freq$. Let T' be any tree representing an optimal prefix code for the alphabet C'. Then the tree T, obtained from T' by replacing the leaf node for z with an internal node having x and y as children, represents an optimal prefix code for the alphabet C.

Proof of Lemma 2

Characters C C’= C – {x,y} + {z}

z.freq = x.freq + y.freq

- Optimal Prefix Tree T’

Proof of Lemma 2

Characters C C’= C – {x,y} + {z}

z.freq = x.freq + y.freq

- Optimal Prefix Tree T ← T’

Proof of Lemma 2

Characters C

C’= C – {x,y} + {z}

z.freq = x.freq + y.freq

- Optimal Prefix Tree T ← T’

How do we show the optimality?

Proof of Lemma 2

- Compare B(T) with B(T’)

Proof of Lemma 2

- Compare B(T) with B(T’)
  - For each character c not equal to x or y?
  -

Proof of Lemma 2

- Compare B(T) with B(T’)
  - For each character c not equal to x or y?
  - Compare the depths of x, y with z.

Proof of Lemma 2

- Compare B(T) with B(T')
  - For each character c not equal to x or y?
  - Compare the depths of x, y with z.

For each character $c \in C - \{x, y\}$, we have that $d_T(c) = d_{T'}(c)$, and hence $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we have

Proof of Lemma 2

- Compare B(T) with B(T')
  - For each character c not equal to x or y?
  - Compare the depths of x, y with z.

For each character $c \in C - \{x, y\}$, we have that $d_T(c) = d_{T'}(c)$, and hence $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we have

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) , \end{aligned}$$

Proof of Lemma 2

- Compare B(T) with B(T')
  - For each character c not equal to x or y?
  - Compare the depths of x, y with z.

For each character $c \in C - \{x, y\}$, we have that $d_T(c) = d_{T'}(c)$, and hence $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we have

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) , \end{aligned}$$

$$B(T) = B(T') + x.freq + y.freq$$

Proof of Lemma 2

Goal: Show that T is optimal for C.

- Compare B(T) with B(T')
  - For each character c not equal to x or y?
  - Compare the depths of x, y with z.

For each character $c \in C - \{x, y\}$, we have that $d_T(c) = d_{T'}(c)$, and hence $c.freq \cdot d_T(c) = c.freq \cdot d_{T'}(c)$. Since $d_T(x) = d_T(y) = d_{T'}(z) + 1$, we have

$$\begin{aligned} x.freq \cdot d_T(x) + y.freq \cdot d_T(y) &= (x.freq + y.freq)(d_{T'}(z) + 1) \\ &= z.freq \cdot d_{T'}(z) + (x.freq + y.freq) , \end{aligned}$$

$$B(T) = B(T') + x.freq + y.freq$$

Proof of Lemma 2

- Proof by contradiction

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T’’ be such an optimal prefix code for C.

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T'' be such an optimal prefix code for C. That is, B(T'') < B(T)

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T'' be such an optimal prefix code for C. That is, B(T'') < B(T)
  - What can we say about x and y in T''?
    -

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T'' be such an optimal prefix code for C. That is, B(T'') < B(T)
  - What can we say about x and y in T''?
    - They are the siblings in T''. **[Greedy Safeness Property]**

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T'' be such an optimal prefix code for C. That is, B(T'') < B(T)
  - What can we say about x and y in T''?
    - They are the siblings in T''.
  - Let T''' be the tree obtained by replacing the parent of x and y with z and
    - z.freq = x.freq + y.freq

Proof of Lemma 2

- Proof by contradiction
  - Suppose that T does not correspond to an optimal prefix code for C.
  - Let T'' be such an optimal prefix code for C. That is, B(T'') < B(T)
  - What can we say about x and y in T''?
    - They are the siblings in T''.
  - Let T''' be the tree obtained by replacing the parent of x and y with z and
    - z.freq = x.freq + y.freq

What is B(T''') in terms of B(T'')?

Proof of Lemma 2

- Let T’’’ be the tree obtained by replacing the parent of x and y with z and
  - z.freq = x.freq + y.freq

$$B(T''') = B(T'') - x.freq - y.freq$$

What is B(T’’’) in terms of B(T’)?

Proof of Lemma 2

- Let T’’’ be the tree obtained by replacing the parent of x and y with z and
  - z.freq = x.freq + y.freq

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T') , \end{aligned}$$

Finish the arguments.....

Proof of Lemma 2

- Let T’’’ be the tree obtained by replacing the parent of x and y with z and
  - z.freq = x.freq + y.freq

$$\begin{aligned} B(T''') &= B(T'') - x.freq - y.freq \\ &< B(T) - x.freq - y.freq \\ &= B(T') , \end{aligned}$$

Contradiction to the assumption that T’ is optimal for C’